

A lighter Docker image ensures that containers spin up faster, saving precious seconds for your company. Also, services like AWS ECR charge you nearly \$0.10/GB of images stored each month, making it a cost-effective choice.

Ways to reduce the Docker image size

There are three different approaches to this task. The first option is to use a third-party service such as Docker Slim to handle the heavy lifting. The next one is to delve deep into Docker images and improve them layer by layer. Lastly, use Docker's own service, which has been recently introduced and simplifies the process of developing Docker images according to best practices.

Docker Slim

Docker Slim (<https://slimtoolkit.org/>) is the best and easiest tool to solve this problem.

- Easily reduces the size of your Docker images.
- Connects to DockerHub and gives you a user-friendly platform for managing images on the registry.
- Checks and fixes vulnerabilities.
- Installation and usage are straightforward.

However, it would be great to see APK and other package installation methods in the future from this platform. As you can see from the results (Figure 1), this service can help you reduce the image size by around 37%.

If you wish to reduce the size of a local Docker image, you can refer to the article by the RedHat Developer Group at https://developers.redhat.com/articles/2022/01/17/reduce-size-container-images-dockerslim#using_dockerslim.

Manual approach

This approach requires a good understanding of your project. There are two steps in this approach, and you can stop after the first step if it satisfies your needs. Let's dig deeper.

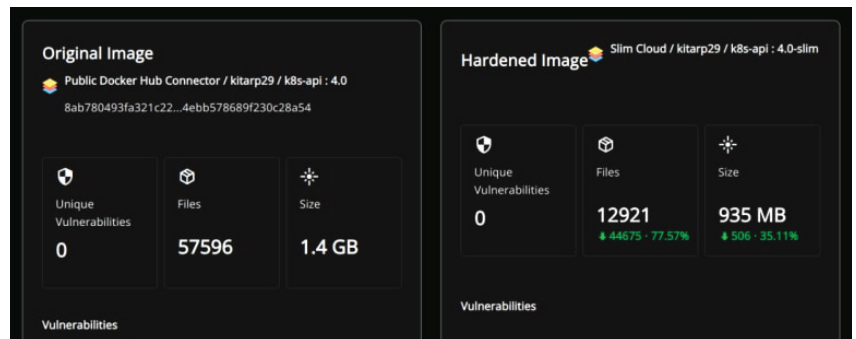


Figure 1: Results using Docker Slim to shrink image size

Using lighter base images: This is the easiest and the first option to consider. Use a lighter image as the base layer for your container. You can opt for an Alpine image of the OS or the programming language you are using.

Alpine images are basic, barebone OS, missing essential software and drivers like graphics, Wi-Fi, and more.

Your DockerFile would look something like this:

```
FROM golang:alpine
WORKDIR /app
COPY . .
RUN go build -o app
CMD ["/app"]
```

Multistage build: The idea is to use two different stages in the image. One is to build the binary, and the next is to simply use it!

In my case, I applied a multistage build pattern, creating a binary in the build stage and using it to run the container. Now the image size is ~35 MB. Further size reduction was achieved by optimising the GCC dependency, resulting in an image size of around 16MB.

Since Go binaries can run independently, I did not need a Go Alpine image.

Here is my final DockerFile:

```
# Build
FROM golang:alpine AS build
WORKDIR /k8-api
COPY . .
RUN go mod download && \
    GO111MODULE=on CGO_ENABLED=0 go \
    build -ldflags "-s -w"
# Deploy
FROM alpine
COPY --from=build /k8-api/k8-api .
ENTRYPOINT ["/k8-api"]
EXPOSE 8000
```

Docker init

Docker has just released a feature called 'docker init', which automates the entire process. It not only reduces the image size but also follows best practices for creating a Docker image. I am still trying it with different projects but the preliminary tests seem promising.

You can read about docker init at <https://docs.docker.com/engine/reference/commandline/init/>

We have discussed three different approaches for reducing Docker image size, and provided tips and resources for further optimisation. You can choose the approach that best suits your needs in order to improve production performance and reliability. **END** 🐧

By: Pratik Singh

The author works at NASDAQ as a senior software developer, and has been dabbling with Go and DevOps for the last two years.